KIT- KALAIGNARKARUNANIDHI INSTITUTE OF TECHNOLOGY

COIMBATORE

DEPARTMENT OF CSE (CYBER SECURITY)

B25ADT303- OBJECT-ORIENTED PROGRAMMING USING JAVA

MINI PROJECT

PROJECT TITLE:

Alumni Database Management System (Java JDBC)

SUBMITTED BY

S.NO	REGISTER NUMBER	STUDENT NAME	TECHNICAL CONTENT (20)	COMPLETENESS (10)	ORIGINALITY (5)	TIMELY SUBMISSION (5)	TOTAL MARKS (40)
1							
2							
3							
4							

MINI PROJECT REPORT

Alumni Database Management System using Java JDBC

B25ADT303 – Object-Oriented Programming using Java

Department of CSE (Cyber Security), KIT – Coimbatore

1 ABSTRACT

An Alumni Database Management System is a software application used by an educational institution to store and retrieve information about former students. This mini project implements such a system in Java using JDBC. The application provides a menu-driven console interface to add, view, search, update and delete alumni records. Each record contains personal, academic and professional details such as name, email, phone, department, batch year, degree, company, designation and city.

The design follows object-oriented principles. An Alumni class encapsulates data, a DAO class performs database operations, a service class validates input, and the main class handles user interaction. JDBC PreparedStatement objects are used for all SQL commands so that the program is safer and easier to maintain. SQLite is used as the database so that the project can be demonstrated without installing a separate database server. A MySQL schema is also provided for laboratories that use MySQL.

2 INTRODUCTION

Colleges need an organised way to keep in contact with alumni. Paper registers and spreadsheets become difficult to search when the number of graduates grows. A database application solves this problem by storing alumni details in a structured table and allowing the user to search by name, department, batch year or company.

This project is developed as part of the course Object-Oriented Programming using Java. It demonstrates classes, objects, encapsulation, layered design, exception handling and JDBC connectivity. The program is suitable for a laboratory demonstration and for a viva on Java, OOP and database connectivity.

3 PROBLEM STATEMENT

Construct an Alumni Database Management System in Java JDBC that can store former-student records and support create, read, update and delete operations through a simple user interface.

4 OBJECTIVES

• To design an alumni record structure using a Java class with private fields and getters/setters.

• To connect a Java application to a relational database using JDBC.

• To implement insert, select, update and delete operations with PreparedStatement.

• To provide search options based on ID, name, department, batch year and company.

• To validate user input such as email, phone number and batch year.

• To apply OOP concepts such as encapsulation, modularity and exception handling.

5 SYSTEM REQUIREMENTS

5.1 Hardware

A personal computer with at least 4 GB RAM and a keyboard for console input is sufficient.

5.2 Software

Component	Details
Operating System	Windows 10 / 11
Language	Java SE (JDK 17 or later; tested on JDK 25)
API	JDBC
Database	SQLite (file alumni.db); MySQL optional
Driver	sqlite-jdbc-3.47.1.0.jar
IDE (optional)	VS Code / IntelliJ / Eclipse / Cursor

6 SOFTWARE ARCHITECTURE

The project uses a layered architecture. Each layer has one responsibility. This makes the code easier to explain in a viva and easier to modify later.

Layer	Class	Responsibility
Presentation	AlumniApp	Shows the menu, reads keyboard input, prints results
Business	AlumniService	Validates data and calls the DAO
Data access	AlumniDAO	Runs SQL using JDBC PreparedStatement
Connection	DBConnection	Opens the SQLite connection and creates the table
Model	Alumni	Stores one alumni record as an object

Control flow: AlumniApp → AlumniService → AlumniDAO → JDBC Driver → SQLite file (alumni.db).

7 OBJECT-ORIENTED CONCEPTS USED

7.1 Class and Object

Alumni, AlumniDAO, AlumniService and AlumniApp are classes. When the program starts, it creates objects such as new AlumniApp() and new AlumniService(). Each alumni row in the database is represented as one Alumni object in memory.

7.2 Encapsulation

Fields of the Alumni class are private. Other classes access them only through getter and setter methods. This hides internal data and prevents accidental modification.

7.3 Modularity / Abstraction

The main class does not write SQL. The DAO does not print menus. Each class exposes only the methods that other classes need. This is abstraction through well-defined interfaces between layers.

7.4 Constructors

The Alumni class has an empty constructor, a constructor without ID (used while inserting) and a constructor with ID (used while reading from the database).

7.5 Exception Handling

SQLException is caught when the database fails. IllegalArgumentException is thrown by the service class when email, phone or year is invalid. The menu loop continues after an error instead of crashing.

8 JDBC OVERVIEW

JDBC (Java Database Connectivity) is the standard Java API used to talk to a relational database. The steps used in this project are:

1. Load / use the JDBC driver (SQLite driver from the lib folder).
2. Open a Connection with DriverManager.getConnection(url).
3. Create a PreparedStatement with an SQL command and ? placeholders.
4. Bind Java values with setString() and setInt().
5. Execute with executeUpdate() for insert/update/delete or executeQuery() for select.
6. Map each ResultSet row into an Alumni object.
7. Close Connection, Statement and ResultSet using try-with-resources.

PreparedStatement is preferred over Statement because user input is sent as parameters, not concatenated into the SQL text. This reduces SQL injection risk and is a good practice to mention in the viva.

9 DATABASE DESIGN

The system uses a single table named alumni. The primary key alumni_id is generated automatically. Email is unique so that the same person is not stored twice.

Column	Type	Constraint	Description
alumni_id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique ID
name	TEXT	NOT NULL	Alumni name
email	TEXT	NOT NULL UNIQUE	Contact email
phone	TEXT	NOT NULL	10-digit mobile number
department	TEXT	NOT NULL	CSE, ECE, IT, MECH, ...
batch_year	INTEGER	NOT NULL	Year of passing
degree	TEXT	NOT NULL	B.E. / B.Tech / ...
company	TEXT	—	Current employer
designation	TEXT	—	Job title
city	TEXT	—	Work / home city

9.1 Sample records (inserted on first run)

ID	Name	Department	Year	Company	City
1	Priya Sharma	CSE	2020	TCS	Chennai
2	Rahul Menon	ECE	2019	Infosys	Bengaluru
3	Ananya Iyer	IT	2021	Wipro	Hyderabad
4	Mohammed Irfan	MECH	2018	L&T	Mumbai

10 MODULES AND WORKING

10.1 Add Alumni

The user enters name, email, phone, department, batch year, degree, company, designation and city. The service checks that fields are not empty, email matches a simple pattern, phone has 10 digits, and year is between 1980 and the current year. A valid record is inserted with INSERT INTO alumni ... VALUES (?,?,?,?,?,?,?,?,?).

10.2 View All Alumni

The program executes SELECT * FROM alumni ORDER BY alumni_id and prints every row in a table format.

10.3 Search

Search by ID uses equality on alumni_id. Search by name and company uses LIKE with a partial match. Search by department and batch year uses exact match. If nothing is found, the program displays a clear message.

10.4 Update Alumni

The user enters an ID. The current record is shown. New values can be typed; pressing Enter without a value keeps the old value for text fields. UPDATE alumni SET ... WHERE alumni_id = ? saves the changes.

10.5 Delete Alumni

Deletion asks the user to type YES. This confirmation reduces accidental removal. DELETE FROM alumni WHERE alumni_id = ? removes the row.

10.6 Total Count

SELECT COUNT(*) FROM alumni reports how many alumni are stored.

11 HOW TO RUN THE PROJECT

Java JDK must already be installed. This project was compiled with javac 25. Open the folder MINI PROJECT in File Explorer. The important files are compile.bat, run.bat, the src folder and the lib folder that contains the SQLite JDBC driver.

Method A — Double-click (easiest)

1. Double-click compile.bat. A window should show “Compile successful.”

2. Double-click run.bat. The alumni menu appears in Command Prompt.

3. Type a number from the menu and press Enter. After you finish, type 0 to exit.

Method B — Command Prompt

Open Command Prompt, then type these commands one by one:

```
cd "C:\Users\Aravindarunachalam\Desktop\opps.java\MINI PROJECT"
compile.bat
run.bat
```

What happens on first run

The program creates a file named alumni.db in the same folder. It also creates the alumni table and inserts four sample records. You can immediately choose option 2 to view them. MySQL is not required for the demo.

If compile fails

If Windows says javac is not recognised, Java is not on PATH. Install JDK and tick the option to set JAVA_HOME, or use a new Command Prompt after installation. The SQLite jar must remain inside the lib folder.

12 HOW TO USE THE APPLICATION

After run.bat starts, this menu is shown:

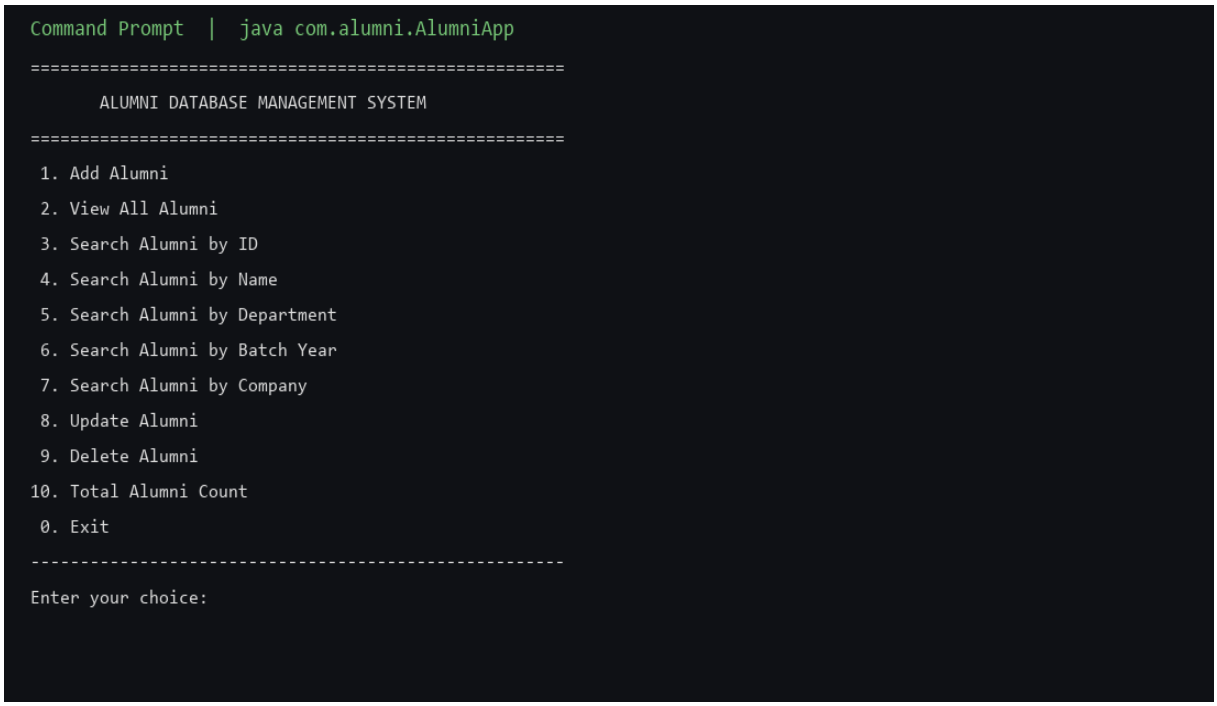


Figure 1. Main menu of the Alumni Database Management System

Choice	What to do	Example input
1 Add	Enter all fields when asked	Name: Aravind, Phone: 9876543210, Year: 2024
2 View all	No extra input	See all rows including sample data
3 Search ID	Type the numeric ID	1
4 Search name	Type full or partial name	Priya
5 Search dept	Type department code	CSE
6 Search year	Type batch year	2020
7 Search company	Type company or part of it	TCS
8 Update	Type ID, then new values	ID 2, change city to Coimbatore
9 Delete	Type ID, then type YES	YES
10 Count	No extra input	Total alumni records: 4
0 Exit	Closes the program	Thank you message

Validation examples: if you enter abc as year, the program asks for a number again. If email has no @ symbol, it prints “Validation error: Enter a valid email address.” If phone is not 10 digits, it rejects the record. Duplicate email values are rejected by the database because email is UNIQUE.

```
Sample Output  |  Menu option 2 - View All Alumni
ID | Name           | Email                 | Dept | Year | Company
---+----------------+-----------------------+------+------+---------
1  | Priya Sharma   | priya.sharma@email.com | CSE  | 2020 | TCS
2  | Rahul Menon    | rahul.menon@email.com  | ECE  | 2019 | Infosys
3  | Ananya Iyer    | ananya.iyer@email.com  | IT   | 2021 | Wipro
4  | Mohammed Irfan | irfan.m@email.com      | MECH | 2018 | L&T
-----------------------------------------------------
Total shown: 4
```

Figure 2. Sample output of View All Alumni

13 SAMPLE TEST CASES

S.No	Test	Expected result
1	View all on first run	Four sample alumni are listed
2	Add valid alumni	Message: Alumni added successfully
3	Add invalid email	Validation error is shown; row is not saved
4	Add phone with 9 digits	Validation error: Phone number must be 10 digits
5	Search name Priya	Priya Sharma record is displayed
6	Search department MECH	Mohammed Irfan record is displayed
7	Update city of ID 1	Record shows the new city
8	Delete with confirmation NO	Delete cancelled; record remains
9	Delete with YES	Alumni deleted successfully
10	Search a missing ID	No alumni found with that ID

14 ADVANTAGES

• Records are stored permanently in alumni.db until deleted.

• Searching is faster than looking through a paper register.

• OOP layering keeps SQL separate from the menu, which is easy to explain.

• Input validation reduces wrong data.

• The project runs without installing MySQL.

15 LIMITATIONS AND FUTURE ENHANCEMENT

The present version is a console application with one table. Login for admin users, a graphical Swing or web interface, photograph storage, event invitations, donation tracking and reports exported to Excel can be added later. The same DAO can be reused if the presentation layer is changed.

16 CONCLUSION

The Alumni Database Management System was designed and implemented in Java using JDBC. The application supports complete CRUD operations, multiple search options and basic validation. It applies class design, encapsulation, layered architecture and exception handling required by an OOP mini project. The working demo with sample data shows that the objectives of the assignment have been met.

17 REFERENCES

1. Herbert Schildt, Java: The Complete Reference, McGraw Hill.

2. Oracle, JDBC Database Access Tutorial.

3. Xerial, SQLite JDBC Driver.

4. B25ADT303 – Object-Oriented Programming using Java, KIT Coimbatore, Mini Project guidelines.

18 APPENDIX A – SOURCE CODE

The complete source is in the src folder of the project. Important files are reproduced below.

A.1 Alumni.java (Model)

```
package com.alumni.model;

public class Alumni {
    private int alumniId;
    private String name;
    private String email;
    private String phone;
    private String department;
    private int batchYear;
    private String degree;
    private String company;
    private String designation;
    private String city;

    public Alumni() {
    }

    public Alumni(String name, String email, String phone, String department, int batchYear,
                  String degree, String company, String designation, String city) {
        this.name = name;
        this.email = email;
        this.phone = phone;
        this.department = department;
        this.batchYear = batchYear;
        this.degree = degree;
        this.company = company;
        this.designation = designation;
        this.city = city;
    }

    public Alumni(int alumniId, String name, String email, String phone, String department,
                  int batchYear, String degree, String company, String designation, String city) {
        this(name, email, phone, department, batchYear, degree, company, designation, city);
        this.alumniId = alumniId;
    }

    public int getAlumniId() {
```

```
    return alumniId;
}

public void setAlumniId(int alumniId) {
    this.alumniId = alumniId;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getPhone() {
    return phone;
}

public void setPhone(String phone) {
    this.phone = phone;
}

public String getDepartment() {
    return department;
}

public void setDepartment(String department) {
    this.department = department;
}

public int getBatchYear() {
    return batchYear;
}

public void setBatchYear(int batchYear) {
    this.batchYear = batchYear;
}

public String getDegree() {
    return degree;
}

public void setDegree(String degree) {
    this.degree = degree;
}

public String getCompany() {
    return company;
}

public void setCompany(String company) {
    this.company = company;
}

public String getDesignation() {
    return designation;
}

public void setDesignation(String designation) {
    this.designation = designation;
}

public String getCity() {
    return city;
}
```

```
    public void setCity(String city) {
        this.city = city;
    }

    @Override
    public String toString() {
        return String.format(
                "ID: %-4d | %-20s | %-24s | %-12s | %-8s | %4d | %-8s | %-16s | %-16s | %s",
                alumniId, name, email, phone, department, batchYear, degree, company, designation, city);
    }
}
```

A.2 DBConnection.java

```
package com.alumni.db;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public final class DBConnection {
    private static final String DB_FILE = "alumni.db";
    private static final String URL = "jdbc:sqlite:" + DB_FILE;

    private DBConnection() {
    }

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL);
    }

    public static void initialize() {
        String createTable = """
                CREATE TABLE IF NOT EXISTS alumni (
                    alumni_id INTEGER PRIMARY KEY AUTOINCREMENT,
                    name TEXT NOT NULL,
                    email TEXT NOT NULL UNIQUE,
                    phone TEXT NOT NULL,
                    department TEXT NOT NULL,
                    batch_year INTEGER NOT NULL,
                    degree TEXT NOT NULL,
                    company TEXT,
                    designation TEXT,
                    city TEXT
                )
                """;

        try (Connection connection = getConnection();
             Statement statement = connection.createStatement()) {
            statement.execute(createTable);
            seedSampleData(statement);
        } catch (SQLException exception) {
            throw new IllegalStateException("Could not initialize the alumni database.", exception);
        }
    }

    private static void seedSampleData(Statement statement) throws SQLException {
        var result = statement.executeQuery("SELECT COUNT(*) FROM alumni");
        if (result.next() && result.getInt(1) == 0) {
            statement.executeUpdate("""
                    INSERT INTO alumni (name, email, phone, department, batch_year, degree, company, designation, cit
                    VALUES
                    ('Priya Sharma', 'priya.sharma@email.com', '9876543210', 'CSE', 2020, 'B.E.', 'TCS', 'Software En
                    ('Rahul Menon', 'rahul.menon@email.com', '9123456780', 'ECE', 2019, 'B.E.', 'Infosys', 'Systems A
                    ('Ananya Iyer', 'ananya.iyer@email.com', '9988776655', 'IT', 2021, 'B.Tech', 'Wipro', 'Developer'
                    ('Mohammed Irfan', 'irfan.m@email.com', '9001122334', 'MECH', 2018, 'B.E.', 'L&T', 'Design Engine
                    """);
        }
    }
}
```

A.3 AlumniDAO.java

```
package com.alumni.dao;

import com.alumni.db.DBConnection;
import com.alumni.model.Alumni;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

public class AlumniDAO {
    public boolean add(Alumni alumni) throws SQLException {
        String sql = """
                INSERT INTO alumni (name, email, phone, department, batch_year, degree, company, designation, city)
                VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                """;
        try (Connection connection = DBConnection.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {
            bindAlumni(statement, alumni);
            return statement.executeUpdate() > 0;
        }
    }

    public List<Alumni> findAll() throws SQLException {
        return query("SELECT * FROM alumni ORDER BY alumni_id", null);
    }

    public Alumni findById(int alumniId) throws SQLException {
        List<Alumni> result = query("SELECT * FROM alumni WHERE alumni_id = ?", alumniId);
        return result.isEmpty() ? null : result.get(0);
    }

    public List<Alumni> findByName(String name) throws SQLException {
        return query("SELECT * FROM alumni WHERE LOWER(name) LIKE ? ORDER BY name",
                "%" + name.toLowerCase() + "%");
    }

    public List<Alumni> findByDepartment(String department) throws SQLException {
        return query("SELECT * FROM alumni WHERE LOWER(department) = ? ORDER BY name",
                department.toLowerCase());
    }

    public List<Alumni> findByBatchYear(int year) throws SQLException {
        return query("SELECT * FROM alumni WHERE batch_year = ? ORDER BY name", year);
    }

    public List<Alumni> findByCompany(String company) throws SQLException {
        return query("SELECT * FROM alumni WHERE LOWER(company) LIKE ? ORDER BY name",
                "%" + company.toLowerCase() + "%");
    }

    public boolean update(Alumni alumni) throws SQLException {
        String sql = """
                UPDATE alumni
                SET name = ?, email = ?, phone = ?, department = ?, batch_year = ?,
                    degree = ?, company = ?, designation = ?, city = ?
                WHERE alumni_id = ?
                """;
        try (Connection connection = DBConnection.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {
            bindAlumni(statement, alumni);
            statement.setInt(10, alumni.getAlumniId());
            return statement.executeUpdate() > 0;
        }
    }

    public boolean delete(int alumniId) throws SQLException {
        String sql = "DELETE FROM alumni WHERE alumni_id = ?";
        try (Connection connection = DBConnection.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setInt(1, alumniId);
            return statement.executeUpdate() > 0;
```

```
        }
    }

    public int count() throws SQLException {
        try (Connection connection = DBConnection.getConnection();
             PreparedStatement statement = connection.prepareStatement("SELECT COUNT(*) FROM alumni");
             ResultSet resultSet = statement.executeQuery()) {
            return resultSet.next() ? resultSet.getInt(1) : 0;
        }
    }

    private void bindAlumni(PreparedStatement statement, Alumni alumni) throws SQLException {
        statement.setString(1, alumni.getName());
        statement.setString(2, alumni.getEmail());
        statement.setString(3, alumni.getPhone());
        statement.setString(4, alumni.getDepartment());
        statement.setInt(5, alumni.getBatchYear());
        statement.setString(6, alumni.getDegree());
        statement.setString(7, alumni.getCompany());
        statement.setString(8, alumni.getDesignation());
        statement.setString(9, alumni.getCity());
    }

    private List<Alumni> query(String sql, Object parameter) throws SQLException {
        List<Alumni> alumniList = new ArrayList<>();
        try (Connection connection = DBConnection.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {
            if (parameter instanceof String value) {
                statement.setString(1, value);
            } else if (parameter instanceof Integer value) {
                statement.setInt(1, value);
            }
            try (ResultSet resultSet = statement.executeQuery()) {
                while (resultSet.next()) {
                    alumniList.add(mapRow(resultSet));
                }
            }
        }
        return alumniList;
    }

    private Alumni mapRow(ResultSet resultSet) throws SQLException {
        return new Alumni(
                resultSet.getInt("alumni_id"),
                resultSet.getString("name"),
                resultSet.getString("email"),
                resultSet.getString("phone"),
                resultSet.getString("department"),
                resultSet.getInt("batch_year"),
                resultSet.getString("degree"),
                resultSet.getString("company"),
                resultSet.getString("designation"),
                resultSet.getString("city")
        );
    }
}
```

A.4 AlumniService.java

```
package com.alumni.service;

import com.alumni.dao.AlumniDAO;
import com.alumni.model.Alumni;

import java.sql.SQLException;
import java.time.Year;
import java.util.List;
import java.util.regex.Pattern;

public class AlumniService {
    private static final Pattern EMAIL = Pattern.compile("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$");
    private static final Pattern PHONE = Pattern.compile("^[0-9]{10}$");
```

```
private final AlumniDAO alumniDAO = new AlumniDAO();

public boolean addAlumni(Alumni alumni) throws SQLException {
    validate(alumni);
    return alumniDAO.add(alumni);
}

public List<Alumni> getAllAlumni() throws SQLException {
    return alumniDAO.findAll();
}

public Alumni getById(int alumniId) throws SQLException {
    return alumniDAO.findById(alumniId);
}

public List<Alumni> searchByName(String name) throws SQLException {
    return alumniDAO.findByName(name.trim());
}

public List<Alumni> searchByDepartment(String department) throws SQLException {
    return alumniDAO.findByDepartment(department.trim());
}

public List<Alumni> searchByBatchYear(int year) throws SQLException {
    return alumniDAO.findByBatchYear(year);
}

public List<Alumni> searchByCompany(String company) throws SQLException {
    return alumniDAO.findByCompany(company.trim());
}

public boolean updateAlumni(Alumni alumni) throws SQLException {
    if (alumniDAO.findById(alumni.getAlumniId()) == null) {
        return false;
    }
    validate(alumni);
    return alumniDAO.update(alumni);
}

public boolean deleteAlumni(int alumniId) throws SQLException {
    return alumniDAO.delete(alumniId);
}

public int totalAlumni() throws SQLException {
    return alumniDAO.count();
}

public void validate(Alumni alumni) {
    requireText(alumni.getName(), "Name");
    requireText(alumni.getEmail(), "Email");
    requireText(alumni.getPhone(), "Phone");
    requireText(alumni.getDepartment(), "Department");
    requireText(alumni.getDegree(), "Degree");
    requireText(alumni.getCompany(), "Company");
    requireText(alumni.getDesignation(), "Designation");
    requireText(alumni.getCity(), "City");

    if (!EMAIL.matcher(alumni.getEmail()).matches()) {
        throw new IllegalArgumentException("Enter a valid email address.");
    }
    if (!PHONE.matcher(alumni.getPhone()).matches()) {
        throw new IllegalArgumentException("Phone number must be 10 digits.");
    }
    int currentYear = Year.now().getValue();
    if (alumni.getBatchYear() < 1980 || alumni.getBatchYear() > currentYear) {
        throw new IllegalArgumentException("Batch year must be between 1980 and " + currentYear + ".");
    }
}

private void requireText(String value, String field) {
    if (value == null || value.isBlank()) {
        throw new IllegalArgumentException(field + " cannot be empty.");
    }
}
```

```
}
```

A.5 AlumniApp.java

```
package com.alumni;

import com.alumni.db.DBConnection;
import com.alumni.model.Alumni;
import com.alumni.service.AlumniService;

import java.sql.SQLException;
import java.util.List;
import java.util.Scanner;

public class AlumniApp {
    private final AlumniService service = new AlumniService();
    private final Scanner scanner = new Scanner(System.in);

    public static void main(String[] args) {
        DBConnection.initialize();
        new AlumniApp().run();
    }

    private void run() {
        boolean running = true;
        while (running) {
            printMenu();
            int choice = readInt("Enter your choice: ");
            System.out.println();
            try {
                switch (choice) {
                    case 1 -> addAlumni();
                    case 2 -> viewAll();
                    case 3 -> searchById();
                    case 4 -> searchByName();
                    case 5 -> searchByDepartment();
                    case 6 -> searchByBatchYear();
                    case 7 -> searchByCompany();
                    case 8 -> updateAlumni();
                    case 9 -> deleteAlumni();
                    case 10 -> showCount();
                    case 0 -> {
                        running = false;
                        System.out.println("Thank you for using Alumni Database Management System.");
                    }
                    default -> System.out.println("Invalid choice. Try again.");
                }
            } catch (IllegalArgumentException exception) {
                System.out.println("Validation error: " + exception.getMessage());
            } catch (SQLException exception) {
                System.out.println("Database error: " + exception.getMessage());
            }
            if (running) {
                System.out.println();
            }
        }
    }

    private void printMenu() {
        System.out.println("======================================================");
        System.out.println("       ALUMNI DATABASE MANAGEMENT SYSTEM");
        System.out.println("======================================================");
        System.out.println(" 1. Add Alumni");
        System.out.println(" 2. View All Alumni");
        System.out.println(" 3. Search Alumni by ID");
        System.out.println(" 4. Search Alumni by Name");
        System.out.println(" 5. Search Alumni by Department");
        System.out.println(" 6. Search Alumni by Batch Year");
        System.out.println(" 7. Search Alumni by Company");
        System.out.println(" 8. Update Alumni");
        System.out.println(" 9. Delete Alumni");
        System.out.println("10. Total Alumni Count");
        System.out.println(" 0. Exit");
```

```
        System.out.println("-----------------------------------------------------");
    }

    private void addAlumni() throws SQLException {
        System.out.println("---- Add Alumni ----");
        Alumni alumni = readAlumniDetails(null);
        if (service.addAlumni(alumni)) {
            System.out.println("Alumni added successfully.");
        } else {
            System.out.println("Could not add alumni.");
        }
    }

    private void viewAll() throws SQLException {
        display(service.getAllAlumni());
    }

    private void searchById() throws SQLException {
        int id = readInt("Enter Alumni ID: ");
        Alumni alumni = service.getById(id);
        if (alumni == null) {
            System.out.println("No alumni found with ID " + id + ".");
        } else {
            printHeader();
            System.out.println(alumni);
        }
    }

    private void searchByName() throws SQLException {
        String name = readLine("Enter name (or part of name): ");
        display(service.searchByName(name));
    }

    private void searchByDepartment() throws SQLException {
        String department = readLine("Enter department (CSE / ECE / IT / MECH / ...): ");
        display(service.searchByDepartment(department));
    }

    private void searchByBatchYear() throws SQLException {
        int year = readInt("Enter batch year: ");
        display(service.searchByBatchYear(year));
    }

    private void searchByCompany() throws SQLException {
        String company = readLine("Enter company: ");
        display(service.searchByCompany(company));
    }

    private void updateAlumni() throws SQLException {
        int id = readInt("Enter Alumni ID to update: ");
        Alumni existing = service.getById(id);
        if (existing == null) {
            System.out.println("No alumni found with ID " + id + ".");
            return;
        }
        System.out.println("Current record:");
        printHeader();
        System.out.println(existing);
        System.out.println("Enter new details:");
        Alumni updated = readAlumniDetails(existing);
        updated.setAlumniId(id);
        if (service.updateAlumni(updated)) {
            System.out.println("Alumni updated successfully.");
        } else {
            System.out.println("Could not update alumni.");
        }
    }

    private void deleteAlumni() throws SQLException {
        int id = readInt("Enter Alumni ID to delete: ");
        Alumni existing = service.getById(id);
        if (existing == null) {
            System.out.println("No alumni found with ID " + id + ".");
            return;
```

```
        }
        System.out.println("Record to delete:");
        printHeader();
        System.out.println(existing);
        String confirm = readLine("Type YES to confirm delete: ");
        if (!"YES".equalsIgnoreCase(confirm.trim())) {
            System.out.println("Delete cancelled.");
            return;
        }
        if (service.deleteAlumni(id)) {
            System.out.println("Alumni deleted successfully.");
        } else {
            System.out.println("Could not delete alumni.");
        }
    }

    private void showCount() throws SQLException {
        System.out.println("Total alumni records: " + service.totalAlumni());
    }

    private Alumni readAlumniDetails(Alumni existing) {
        String name = prompt("Name", existing == null ? null : existing.getName());
        String email = prompt("Email", existing == null ? null : existing.getEmail());
        String phone = prompt("Phone (10 digits)", existing == null ? null : existing.getPhone());
        String department = prompt("Department", existing == null ? null : existing.getDepartment());
        int batchYear = readInt("Batch year" + (existing == null ? ": " : " [" + existing.getBatchYear() + "]: "));
        if (existing != null && batchYear == 0) {
            batchYear = existing.getBatchYear();
        }
        String degree = prompt("Degree", existing == null ? null : existing.getDegree());
        String company = prompt("Company", existing == null ? null : existing.getCompany());
        String designation = prompt("Designation", existing == null ? null : existing.getDesignation());
        String city = prompt("City", existing == null ? null : existing.getCity());
        return new Alumni(name, email, phone, department, batchYear, degree, company, designation, city);
    }

    private String prompt(String label, String current) {
        if (current == null) {
            return readLine(label + ": ");
        }
        String value = readLine(label + " [" + current + "]: ");
        return value.isBlank() ? current : value;
    }

    private void display(List<Alumni> alumniList) {
        if (alumniList.isEmpty()) {
            System.out.println("No alumni records found.");
            return;
        }
        printHeader();
        for (Alumni alumni : alumniList) {
            System.out.println(alumni);
        }
        System.out.println("-------------------------------------------------------");
        System.out.println("Total shown: " + alumniList.size());
    }

    private void printHeader() {
        System.out.println(String.format(
                "%-9s | %-20s | %-24s | %-12s | %-8s | %-4s | %-8s | %-16s | %-16s | %s",
                "ID", "Name", "Email", "Phone", "Dept", "Year", "Degree", "Company", "Designation", "City"));
        System.out.println("-".repeat(160));
    }

    private String readLine(String prompt) {
        System.out.print(prompt);
        return scanner.nextLine().trim();
    }

    private int readInt(String prompt) {
        while (true) {
            System.out.print(prompt);
            String input = scanner.nextLine().trim();
            if (input.isEmpty()) {
```

```
                return 0;
            }
            try {
                return Integer.parseInt(input);
            } catch (NumberFormatException exception) {
                System.out.println("Enter a valid number.");
            }
        }
    }
}
```

19 APPENDIX B – PROJECT FOLDER

```
MINI PROJECT
  compile.bat
  run.bat
  alumni.db                 (created after first run)
  lib\sqlite-jdbc-3.47.1.0.jar
  sql\schema_mysql.sql
  src\com\alumni\AlumniApp.java
  src\com\alumni\model\Alumni.java
  src\com\alumni\db\DBConnection.java
  src\com\alumni\dao\AlumniDAO.java
  src\com\alumni\service\AlumniService.java
  Alumni_Database_Management_System_Report.docx
  Alumni_Database_Management_System_Report.pdf
```